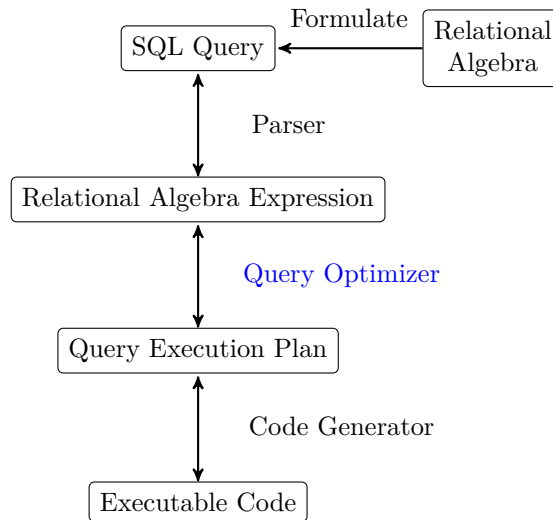


Query Optimization

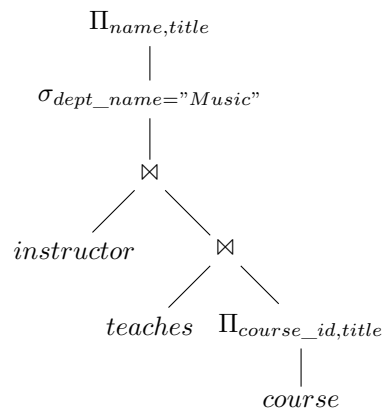
Weihai Yu

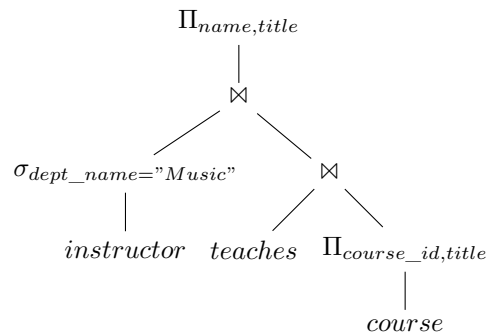
October 17, 2019

Query optimization



Equivalent expressions

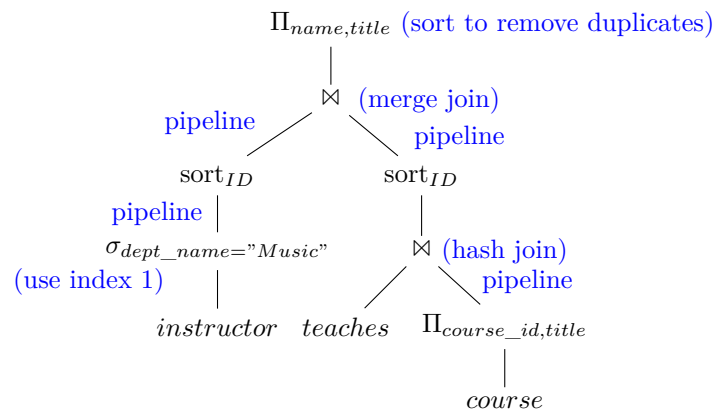




In the second tree, σ is pushed. This is the most common heuristic.

Not pushing it, as in the first tree, may also be useful. For example, when the index is used in $\bowtie$.

Execution plan



Try in SQLite:

```
explain query plan
select distinct c.customernumber, c.customername
from customers c
where not exists
  (select distinct d1.productcode
   from orders o1, orderdetails d1
   where o1.ordernumber = d1.ordernumber
     and o1.customernumber=219
  except
   select distinct d2.productcode
   from orders o2, orderdetails d2
   where c.customernumber = o2.customernumber
     and o2.customernumber != 219
     and o2.ordernumber = d2.ordernumber);
```

and see what happens.

Equivalent expressions

1. $\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$
2. $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$
3. $\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_1}(E)$
4.
 - $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$
 - $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$
5. $E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$
6. $(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$

Equivalent expressions

7. $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1} E_1) \bowtie_{\theta} (\sigma_{\theta_2} E_2)$
when θ_1 involves only the attributes in E_1 and θ_2 involves only the attributes in E_2
8. $\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = \Pi_{L_1 \cup L_2}(\Pi_{L_1 \cup L_3}(E_1) \bowtie_{\theta} \Pi_{L_2 \cup L_4}(E_2))$
attributes
 - L_1, L_2 : from E_1, E_2
 - L_3 from E_1 and θ
 - L_4 from E_2 and θ

Choosing an expression

- Choosing the cheapest algorithm for each operation independently may not yield best overall algorithm
 - merge-join may be costlier than hash join, but provides sorted output that reduces the cost of the subsequent operation
- two approaches
 - compare the cost of different plans and choose a best one
 - use heuristics

Cost-based optimization

- we have discussed the cost of each operation earlier
- statistics on input data (database system catalog) and estimation of expression results
- consider $r_1 \bowtie r_2 \bowtie \dots \bowtie r_n$
- $\frac{(2(n-1))!}{(n-1)!}$ different orders
 - 665280 when $n = 7$
 - 176 billion when $n = 10$
- reduce the number of expressions
 - heuristics

Heuristic optimization

- perform selections early (sometimes not, when?)
- perform projections early (sometimes not, when?)
- perform selective operations early
- restrict to particular tree shapes

